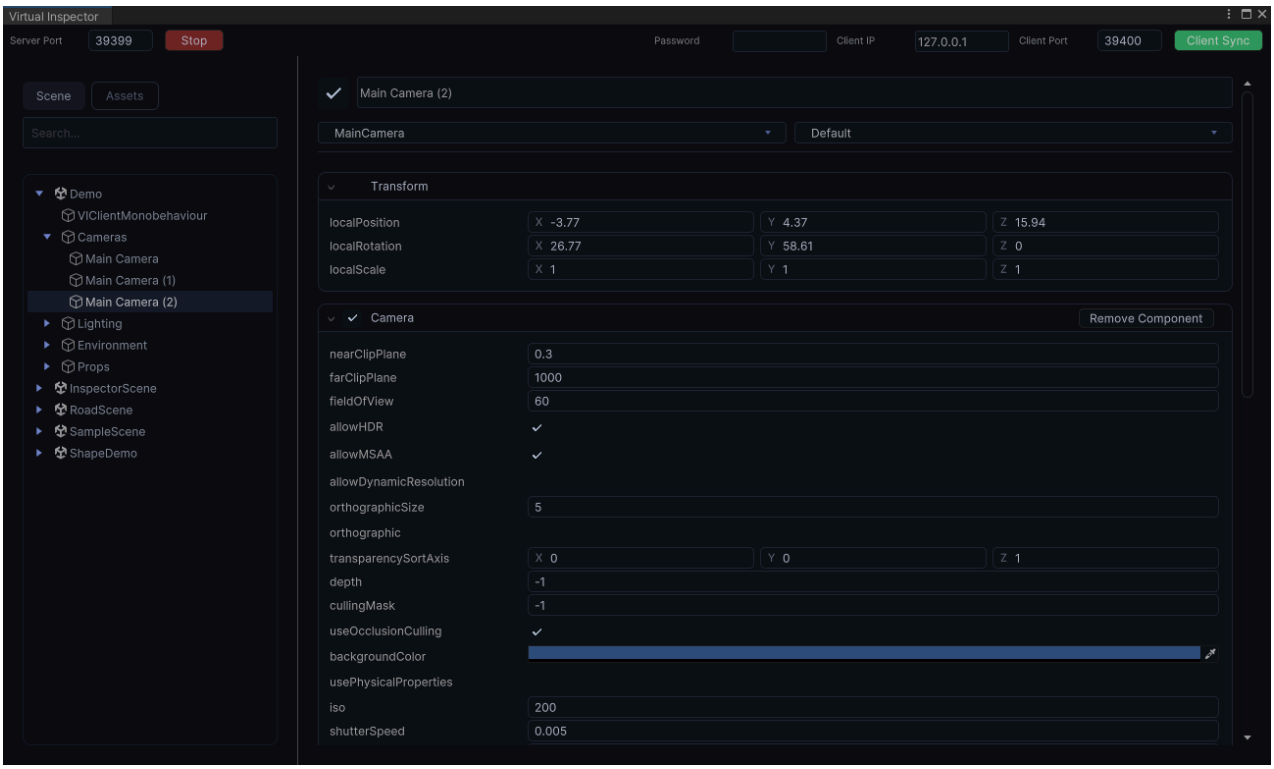
Documentation

Getting Started

Welcome



Welcome to the official documentation of **Virtual Inspector** — a cross-device debugging and inspection solution for any Unity project.

Virtual Inspector allows you to **view, edit, and synchronize your application in real time**, whether it's a game, simulation, prototype, or production tool.
No rebuilds, no restarts, no interruption of gameplay or execution.

With Virtual Inspector, you can inspect objects, modify components, browse assets, and monitor runtime changes directly from another device — on PC, mobile, or tablet.

> ⚡ *Your Unity project, fully visible and interactive in real time.*

Use this documentation to get started quickly, explore the main features, and integrate Virtual Inspector into your workflow.

Quickstart

What is Virtual Inspector?

Virtual Inspector is a real-time debugging and inspection tool for Unity applications and games.
It allows you to explore your scenes, inspect objects, modify components, and analyze runtime data directly from another device — without rebuilding your project or restarting the application.

Designed for both developers and QA teams, Virtual Inspector drastically reduces iteration time by giving you **live access** to your runtime environment, whether running on **PC, mobile, consoles, VR, or standalone builds**.

What can you do with Virtual Inspector?

With Virtual Inspector, you can:

- View your entire scene hierarchy in real time
- Inspect and edit components, materials, and assets
- Modify transforms, properties, and settings instantly
- Detect instantiated assets and track runtime changes
- Collaborate across devices (PC, mobile, tablet)
- Debug builds without needing the Unity Editor

> ⚡ *No rebuilds. No relaunches. Instant debugging.*

Why use it?

- **Save hours** by eliminating unnecessary builds
- Instantly understand what is happening inside your runtime
- Debug on devices where the Unity Editor cannot run

- Improve team collaboration and visibility
- Iterate dramatically faster during development and playtesting

Installation

⚙️ Installation

Virtual Inspector is composed of **three modules**, all included in the same Unity package:

- **Client** — installed in the Unity project you want to inspect
- **Server** — the interface that displays hierarchy, components, and live data
- **Common** — shared systems used by both Client and Server

These modules must remain together and follow the correct folder structure.

📦 1. Importing the Package

1. Download the Virtual Inspector Unity package (`.unitypackage`).
2. Open your Unity project.
3. Go to **Assets → Import Package → Custom Package...**
4. Select the Virtual Inspector package.
5. Import all required files.

After import, you will see three folders:

- **VirtualInspector/Client**
- **VirtualInspector/Server**
- **VirtualInspector/Common**

2. Understanding the Folder Structure

Client

Contains all runtime systems used inside the inspected Unity project.

Server

Contains the editor and UI tools used to inspect and modify the running application.

Common

Contains shared classes, data structures, serialization logic, and network protocol used by *both* the Client and Server.

> ⚠️ **Important:**
> The **Common** folder is required in **every project** using Virtual Inspector — whether it's an inspected project, a server project, or both.

Without the Common module, Client and Server cannot communicate.

Installing the Client (Required)

You must install the **Client** module in the Unity project you want to inspect.

The Client is responsible for:

- Capturing scene hierarchy
- Tracking components, transforms, and assets
- Sending updates to the Server
- Applying any live edits received

✅ Steps

1. Ensure **VirtualInspector/Client** and **VirtualInspector/Common** are present.
2. Add the `VIClientMonobehaviour` component to a GameObject in your startup scene.
   - Recommended: create an empty object named **VirtualInspectorClient**
3. Configure connection settings if needed. (See Project Parameter)

Once running, the Client starts sending data to the Server automatically.

Installing the Server (Flexible)

The **Server** can be installed in two ways depending on your workflow.

Option A — Use the Server inside the same project

(Simplest setup)

You keep all three folders:

```
VirtualInspector/Client
VirtualInspector/Server
VirtualInspector/Common
```

This allows you to:

- Run your scene
- Open the Server window
- Debug instantly inside the same project

Ideal during development.

Option B — Install the Server in a separate empty project

(Recommended for teams)

You may want to **remove the Server** from your inspected project and place it in a **dedicated Server Workspace Project**.

Why?

- Keep your main project clean
- Let QA or teammates inspect builds without opening your game project

- Debug mobile/console devices from a lightweight editor project
- Avoid shipping editor-only files inside production projects

✅ Required folder structure in the Server project

In the **dedicated Server project**, you should keep:

```
VirtualInspector/Server
VirtualInspector/Common
```

And delete:

```
VirtualInspector/Client
```

✅ Required folder structure in the Client project

In your **inspected Unity project**, you must keep:

```
VirtualInspector/Client
VirtualInspector/Common
```

And optionally delete:

```
VirtualInspector/Server
```

> The **Common** folder must exist in both projects to ensure protocol compatibility.

Changelog

Version 1.0.0 — Initial Release

First public release of **Virtual Inspector**.

Core Features

- Runtime hierarchy inspection (Scenes & GameObjects)
- Live synchronized hierarchy updates
- Runtime inspector with editable fields
- Client / Server architecture for remote inspection
- Support for production builds and external devices

Networking & Remote Workflow

- Client / Server communication over **local network (LAN)**
- Devices must be connected to the **same Wi-Fi or wired network**
- Optimized for low-latency local communication
- Ideal for PC, mobile devices, consoles, and embedded targets

Inspector System

- Component cards with fold / unfold
- Enable / disable components at runtime
- Field support:
  - Bool
  - Int
  - Float
  - String
  - Vector2 / Vector3 / Vector4
  - Color

    - Object references
- Editable arrays (resize, reorder, per-element editing)
- Automatic struct & path grouping

Component Management

- Add components at runtime
- Searchable component tree
- Remove components safely

Hierarchy & Context Menus

- Create / Duplicate / Delete GameObjects
- Rename objects
- Context-aware hierarchy actions

User Interface

- Fully redesigned UI Toolkit interface
- Dark theme optimized for long sessions
- Custom fields and compact layouts

Tools & Workflow

- Dedicated Launcher window
- Quick access to documentation and links
- Platform-safe runtime inspection
- Designed for devices where Unity Editor cannot run

Configuration

Setup

After installing Virtual Inspector, a short configuration step is required to ensure the Client and Server communicate correctly and that all assets, tags, layers, and project metadata are properly synchronized.

This setup involves:

1. Configuring the **Project Parameter** ScriptableObject
2. Building the **Project Manifest**

Both are essential for Virtual Inspector to operate efficiently.

✅ Setup Summary

Step	Description
1. Configure Project Parameter	Defines ports, send frequency, and asset syncing behavior
2. Build Project Manifest	Generates metadata and references for assets, tags, layers, etc.
3. Run Client + Server	Connection establishes automatically

Project Parameter

The **Project Parameter** defines all runtime network settings and synchronization rules used by the Client.
It is a **ScriptableObject** located here:

```
VirtualInspector/Client/Resources/VirtualInspector/ProjectParameter.asset
```

This asset is automatically loaded by the Client at runtime.

Editable Fields

Client Listen Port

The UDP port the Client uses to receive messages from the Server.

- Default: `39400`
- Must match the port configured in the Server window
- Change this if another tool already uses this port on your machine

---

Send Hz

The frequency (per second) at which the Client sends synchronization packets.

Example:
`Send Hz = 10` → send updates every 0.1 seconds.

Higher values increase responsiveness but also increase bandwidth usage.

Max Send Packet By Frame

Limits how many packets can be sent during a single frame.

- Prevents spikes
- Useful on mobile or low-power devices

Max Send Packets By Second

Global bandwidth limiter for the Client.

- Helps avoid flooding the network
- Ensures stable performance on wireless connections

Building the Project Manifest

The **Project Manifest** is required for Virtual Inspector to understand your project structure.
It contains:

- Project metadata such as:
  - Tags
  - Layers
  - Scenes Description (Stable 64-bit ID, GUID, Path, Name)
  - Sorting layers
  - Components Metadata (Type name, CategoryPath)

This manifest allows the Server to fully reconstruct and interpret your project environment.

Custom Component Categories

Virtual Inspector allows you to define your own **component categories**, icons, and display organization.

These definitions come from the XML file:

```
Assets/VirtualInspector/Client/Resources/VirtualInspector/predefined_co
mponents.xml
```

Inside this file, you can:

- Create new categories
- Assign icons (icons are stored but **not used yet**) sx
- Register custom components
- Organize components into groups
- Override or extend default definitions

Example structure:

```
<ComponentCategories>
    <Category name="Audio" icon="Assets/Icons/audio.png">
        <Component name="Audio Source" type="UnityEngine.AudioSource"
/>
        <Component name="Audio Listener"
type="UnityEngine.AudioListener" />
    </Category>

    <Category name="My Custom Tools" icon="Assets/Icons/custom.png">
        <Component name="Health Component" type="MyGame.Health" />
        <Component name="AI Controller" type="MyGame.AIController" />
    </Category>
</ComponentCategories>
```

Why build the Manifest?

Virtual Inspector uses this manifest to:

- Correctly map **tags** and **layers**
- Identify and name scenes
- Categorize components
- Display consistent metadata on the Server

Without it, the Server cannot resolve project-level information.

How to build it

Inside Unity, open:

```
Tools → Virtual Inspector → Build Project Manifest
```

This will generate or update the manifest asset inside:

```
VirtualInspector/Client/Resources/VirtualInspector/ProjectManifest.asse
t
```

The Client automatically loads this manifest on startup.

When to rebuild

Rebuild the manifest whenever you change:

- Tags
- Layers
- Scenes (add, rename, move, build index)
- Component XML definitions
- Scripts are added, removed, or renamed (affects component registration)

Automatic Build (Optional)

You can enable **automatic manifest generation** during Unity builds.

In the **ProjectParameter** asset, enable:

```
buildManifestOnBuildProject
```

If enabled, the Manifest will be rebuilt **every time the project is built** (e.g. Build & Run).

Client Password & Secret Key

Virtual Inspector supports an optional authentication system to secure connections between the Server and the Client.
Authentication is handled using a **challenge–response mechanism** based on **HMAC-SHA256**, meaning the actual password or secret key is **never transmitted over the network**.

Password (Manual User Input)

You can type a temporary password directly in the Server toolbar.
When the Client connects, it will challenge the Server, and authentication succeeds only if both sides use the same password.

Characteristics:

- The password is **not stored** anywhere
- It **resets when Unity closes**
- Only a **hashed HMAC** is exchanged, never the password itself
- Ideal for **quick or temporary protected sessions**

Client Secret (Auto-Generated Authentication Key)

For more secure or automated setups, you can use a **Client Secret** instead of entering a password manually.

✅ What is the Client Secret?

A unique cryptographic identifier generated inside the inspected Unity project.
It is used to authenticate the Server **automatically**, without requiring user input each time.

✅ How it works

1. Open the **Project Parameter** ScriptableObject
2. Enter your password and Click **Generate Client Secret**
3. A unique hashed key + salt are created and stored in the Client configuration
4. When connecting, the Client challenges the Server
5. The Server computes an HMAC-SHA256 response using the same secret
6. If the hash matches → authentication succeeds
7. If not → the connection is rejected

✅ **Recommended for:**

- Teams working on shared projects
- Debugging over Wi-Fi or LAN networks
- External testers or QA pipelines
- Console deployments (Switch, PlayStation, etc.)

Connection Requirements

Connection Requirements & Limitations

For a stable communication between the Virtual Inspector **Server** and **Client**, several conditions must be met.
Failing to meet these conditions can result in lost data, failed updates, or connection drops.

1. Server & Client must remain active during the entire session

Virtual Inspector relies on continuous, real-time UDP communication.
If either the Server or the Client becomes inactive, minimized, suspended, or paused:

- Some messages may be **lost**
- Component updates may fail
- The hierarchy may become outdated
- The session may require a manual restart

✓ Recommended:

- Run the Client on a separate device
- Or enable **Run in Background** in Unity
  (Edit → Project Settings → Player → Resolution and Presentation → Run in Background)
- Avoid pausing or switching away from the Client during inspection
- Always keep the Server window open inside the Editor

If this happens:

> **Stop the connection, then Start again**
> This forces a clean handshake and resynchronization.

2. Mobile data / 4G is NOT supported

Virtual Inspector uses **local UDP communication**, which requires both the Server and Client to be connected on the **same local network**.

❌ Not supported

- 4G / 5G mobile data
- Mobile hotspots (unless they act as a real LAN, which is rare)
- Cellular networks with NAT
- Internet/WAN connections

These connections block or reroute UDP traffic, preventing the handshake and authentication from working correctly.

✓ Supported connection types

- Local Wi-Fi network
- Local Ethernet network
- Emulator or localhost (`127.0.0.1`)
- Console devkits on the same LAN

Interface

ToolBar

The toolbar is located at the top of the Virtual Inspector window.
It contains all the controls needed to connect the Server to the Client.

Toolbar fields

Server Port

The port used by the Server to listen for incoming Client connections.
Must match the **Client Listen Port** defined in the Project Parameter asset.

Start / Stop

Starts or stops the Server instance running inside the Unity Editor.

Password (Optional)

A temporary password you can set to protect the connection.
This is **not stored anywhere** and resets when Unity restarts.

Useful for local sessions where you want manual access control.

Client IP

The IP address of the device running the Client.

Examples :

- Local computer → `127.0.0.1`
- Mobile device → your LAN IP
- Console / build device → IP displayed by your system

Client Port

The port on which the Client listens for Server messages.

Must match the **Client Listen Port** configured in the Project Parameter.

Client Sync (ON / OFF)

Controls synchronization of runtime data from the Client to the Server.

- **ON** → Runtime values are continuously sent from the Client to the Server.
  The Server inspector reflects live changes happening in the running application.
- **OFF** → Runtime data is no longer sent from the Client to the Server.
  The Server can still modify values, and changes are applied to the Client as usual.

This is useful for:

- Pausing live data updates while inspecting values
- Preventing noise or unintended runtime changes
- Demonstrations or controlled debugging sessions

Hierarchy Panel

The left section of the interface displays a **live synchronized hierarchy**.

It contains **two tabs**:

① **Scenes**

Shows all loaded scenes and their full GameObject hierarchy.

- Expands/collapses like Unity's standard hierarchy
- Updates in real time when GameObjects appear or disappear
- Displays scene root objects, children, and nested structures
- Shows persistent scenes such as `DontDestroyOnLoad`

② Assets

The **Assets** tab does *not* display all project assets.
Currently, it shows **only the assets that are currently loaded in memory** while your application is running.

Virtual Inspector tracks a limited set of runtime asset types:

- **Materials**
- **ScriptableObjects**
- **Textures**
- **Texture2D**

These assets appear automatically when Unity loads them (e.g., when a scene loads) and disappear when they are unloaded.

Search Bar

Filters objects by name across the active tab.

Context Menu

Right-clicking inside the Hierarchy opens a contextual menu that allows you to create, duplicate, or delete GameObjects inside the connected client application.

There are **two types of context menus**:

- **Right-click on empty space** → create objects at root level
- **Right-click on an entity** → create child objects + object actions (Duplicate, Delete)

Inspector Panel

The Inspector (right section) displays detailed information for the selected item:

- GameObject properties
- Transform
- Components
- Materials
- Custom components
- SerializeField variables
- Runtime values (float, int, bool, vector, color...)

The Inspector is fully interactive when **Live Edit** is enabled:

- Editing a field instantly updates the running application
- Components can be enabled/disabled

- Transforms can be modified in real time
- Materials and properties update immediately on the Client device

Each component panel mirrors Unity's original Inspector structure but synchronized through the Virtual Inspector network protocol.

Supported vs Unsupported Types

Category	Support
Primitive fields (int, float, bool, string)	✅
Unity types (Vector2/3/4, Quaternion, Color)	✅
Serializable structs & classes	✅
UnityEngine.Object references	✅
Arrays of supported types	✅
Properties (public or private) with getter & setter	✅
Enums (scalar or flags)	❌
Lists (`List<T>`)	❌
Dictionaries	❌
Static fields	❌
Read-only properties (`get` only)	❌
Unsupported containers (HashSet, Queue, Stack...)	❌
Private fields not serialized	❌

Adding Components

You can add new components to the selected GameObject using the **Add Component** button.

The searchable dropdown includes:

- **Predefined components** listed in `predefined_components.xml`
- **Custom MonoBehaviours** from your project scripts
- **Unity built-in components** derived from `Component`
- All entries are **organized into categories** based on your Project Manifest

Example categories:

- Animation
- Physics

- Rendering
- UI
- Audio
- Scripts

Selecting a component instantly adds it to the target GameObject in the client application.

Connection Status Messages

Connection Status Messages

Virtual Inspector displays connection feedback messages at the top of the Server window.
These messages indicate whether the Client was found, authenticated, or rejected.

Understanding these messages helps diagnose connection, password, and network issues.

🟢 Successful Messages

Client authentication SUCCESS.

This means:

- The Server successfully contacted the Client
- The Client responded
- The password or Client Secret matched
- The handshake is complete
  → The connection is fully established.

🔴 Error Messages

❗ Client not found — no response received.

Shown when:

- The Server sends a HELLO packet
- The Client does *not* respond in time

Possible causes:

- Wrong **Client IP**
- Wrong **Client Port** (The port used by the Client is defined in the ProjectParameter asset inside the inspected Unity project.)
- Client application not running
- Firewall or OS security blocking UDP traffic
- Client crashed or stuck during initialization
- Another process already uses this port on the device

❗ Client authentication FAILED. Closing server session.

Shown when:

- A Client responded
- BUT the authentication hash did not match

Possible causes:

- Wrong password
- Wrong Client Secret

- Server & Client do not share the same secret
- Leftover salt or secret mismatch after regeneration

The Server automatically closes the session to avoid unauthorized access.